BỘ CÔNG THƯƠNG – DỰ ÁN CHUYỂN ĐỔI SỐ DOANH NGHIỆP
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG (ICTU)

o0o

TÀI LIỆU KỸ THUẬT
HƯỚNG DẪN XÂY DỰNG CHATBOT AI DOANH NGHIỆP THEO KIẾN TRÚC RAG (RETRIEVAL-AUGMENTED GENERATION)

Chuẩn hóa kiến trúc 6 thành phần, quy trình triển khai 4 bước chuẩn mẫu (Template), khung Prompt điều khiển AI Agent và minh chứng thực nghiệm dự án One Store

Phân loại tài liệu:	Tài liệu Hướng dẫn Kỹ thuật & Thực hành Triển khai (Technical Guidelines)
Đối tượng áp dụng:	Kỹ sư AI, Kỹ sư Full-stack, Trưởng nhóm Công nghệ, Giảng viên & Sinh viên CNTT
Dự án tham chiếu thực tế:	One - Store (Hệ thống phân phối sneaker thể thao chính hãng Âu - Á)
Phiên bản phát hành:	v2.0 — Cập nhật chuẩn hóa tháng 11/2026 (Zero-Emoji, Clean Code)

MỤC LỤC TỔNG QUAN TÀI LIỆU

PHẦN I. CẤU TRÚC CHATBOT AI DOANH NGHIỆP	1.1 Mục tiêu kiến trúc & Giải quyết bài toán ảo giác 1.2 Chi tiết 6 thành phần cốt lõi của hệ thống Chatbot RAG 1.3 Sơ đồ kiến trúc tổng thể & Cơ chế luồng dữ liệu hai tầng
PHẦN II. QUY TRÌNH TRIỂN KHAI CHATBOT AI (TEMPLATE)	2.1 Sơ đồ quy trình 4 bước tổng quát (Template cho mọi dự án) 2.2 Bước 1: Chuẩn bị tri thức và chia đoạn văn bản (Smart Chunking) 2.3 Bước 2: Vector hóa và nạp dữ liệu (Embedding & Indexing Pipeline) 2.4 Bước 3: Xây dựng Pipeline RAG Backend (Retrieval + Generation) 2.5 Bước 4: Tích hợp Giao diện Chat Widget và Kiểm thử hệ thống
PHẦN III. SỬ DỤNG PROMPT ĐỂ XÂY DỰNG CHATBOT AI	3.1 Prompt Bước 1: Chuẩn bị tri thức và chia đoạn (Tổng quát & One Store) 3.2 Prompt Bước 2: Vector hóa và nạp dữ liệu (Tổng quát & One Store)

	3.3 Prompt Bước 3: Xây dựng Pipeline RAG Backend (Tổng quát & One Store) 3.4 Prompt Bước 4: Tích hợp Giao diện & Kiểm thử (Tổng quát & One Store) 3.5 Nguyên tắc thiết kế Prompt điều khiển AI Agent hiệu quả
PHẦN IV. KẾT QUẢ TRIỂN KHAI CHATBOT AI (ONE STORE)	4.1 Giao diện Chatbot thực tế trên website One Store 4.2 Kết quả truy vấn và trả lời thực tế (Q&A Demo) 4.3 Kết quả kiểm thử hệ thống & API Backend 4.4 Bảng tổng kết nghiệm thu hệ thống

PHẦN I. CẤU TRÚC CHATBOT AI DOANH NGHIỆP

Mục tiêu phần này giúp người đọc nắm vững bản chất kiến trúc của một hệ thống Chatbot AI ứng dụng kỹ thuật **RAG (Retrieval-Augmented Generation — Tạo sinh tăng cường truy xuất tri thức)**. Khác với các chatbot truyền thống chỉ dựa vào bộ nhớ tĩnh đóng băng của mô hình ngôn ngữ lớn, kiến trúc RAG tích hợp kho dữ liệu sống động của doanh nghiệp để đưa ra câu trả lời chuẩn xác và có thể kiểm chứng.

1.1. Giải quyết bài toán ảo giác (Hallucination) trong doanh nghiệp

Trong môi trường kinh doanh thực tế (thương mại điện tử, tài chính, bán lẻ), việc mô hình AI tự suy diễn thông tin sai lệch về giá bán, thông số sản phẩm hoặc chính sách bảo hành có thể dẫn đến thiệt hại nghiêm trọng. Cần lưu ý một nguyên tắc kỹ thuật cốt lõi:

> ⚠ **Lưu ý quan trọng:** *Kiến trúc RAG không tuyên bố loại bỏ hoàn toàn 100% hiện tượng ảo giác (hallucination). RAG giúp giảm thiểu tối đa nguy cơ trả lời sai bằng cách bắt buộc mô hình chỉ suy luận dựa trên các trích đoạn văn bản thực tế được truy xuất từ nguồn tri thức có căn cứ, đồng thời đính kèm nguồn tham khảo (Sources) minh bạch.*

1.2. Sáu thành phần cốt lõi của hệ thống Chatbot AI

Hệ thống Chatbot RAG doanh nghiệp hoàn chỉnh được cấu thành từ 6 thành phần chuyên biệt:

- **1. Giao diện người dùng (UI Widget):** Cửa sổ chat nổi (Floating Widget) hoặc thanh hội thoại tích hợp trực tiếp trên web/app. Đảm nhiệm chức năng nhận câu hỏi, tự động cuộn (auto-scroll), hiển thị trạng thái đang nhập (typing indicator) và hiển thị thẻ trích dẫn nguồn tham khảo để người dùng đối chiếu.
- **2. Cổng điều phối (Backend API Gateway):** API Server tiếp nhận request qua endpoint chuẩn (ví dụ POST /api/chat). Đóng vai trò 'nhạc trưởng' kiểm duyệt an toàn (Guardrails, chống Prompt Injection/Spam), điều phối tuần tự quy trình RAG và chuẩn hóa dữ liệu JSON phản hồi.
- **3. Kho dữ liệu gốc (Knowledge Base):** CSDL quan hệ (SQL/NoSQL) lưu trữ toàn văn các văn bản chính sách, hồ sơ cửa hàng, bảng quy đổi size và danh mục sản phẩm kèm metadata phân loại (ví dụ SQLite D1, PostgreSQL, MySQL).
- **4. Mô hình nhúng (Embedding Model):** Mô hình AI chuyên sâu chuyển đổi các đoạn văn bản ngôn ngữ tự nhiên thành mảng số vector toán học đa chiều (ví dụ vector 1024 chiều), bảo toàn trọn vẹn ngữ nghĩa để phục vụ việc so khớp toán học (ví dụ @cf/baai/bge-m3, text-embedding-3-small).
- **5. Cơ sở dữ liệu Vector (Vector Database):** Hệ thống lưu trữ vector số chuyên dụng, hỗ trợ thuật toán quét độ tương đồng hình học (Cosine Similarity, Euclidean Distance) với độ trễ cực thấp (< 35ms) để trích xuất Top-K đoạn tri thức gần nhất với câu hỏi (ví dụ Cloudflare Vectorize, Pinecone, Qdrant).

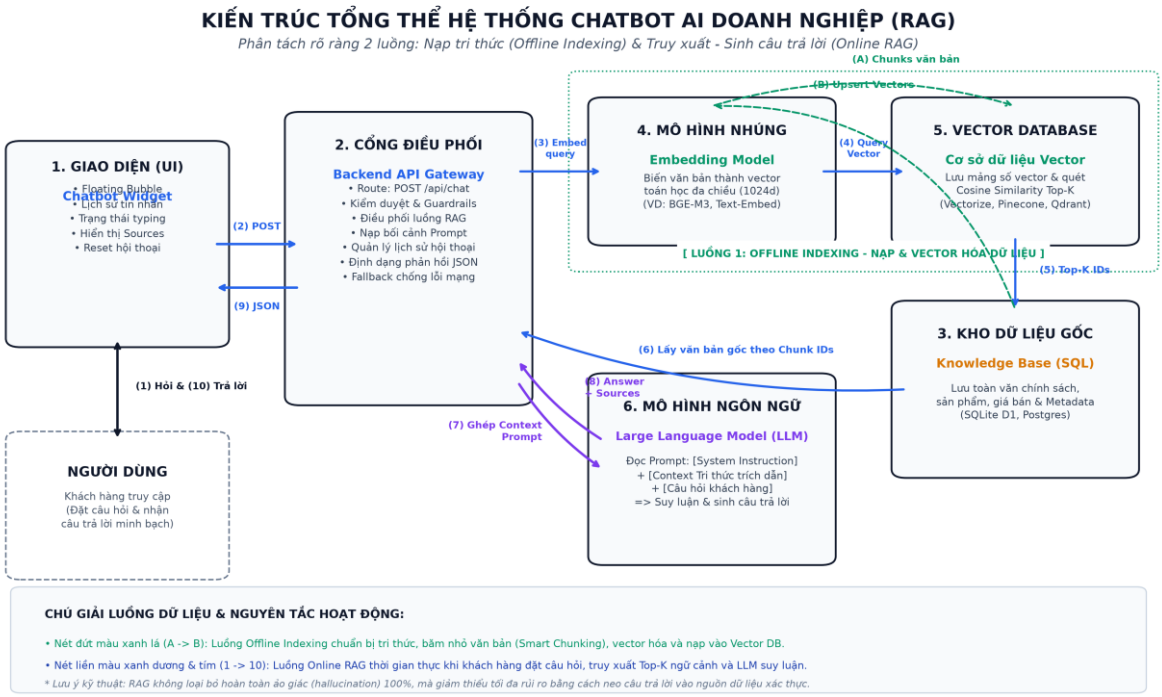
- **6. Mô hình ngôn ngữ lớn (LLM):** Mô hình AI tạo sinh nhận đầu vào là Prompt được đóng gói gồm [Chỉ dẫn vai trò chống ảo giác] + [Ngữ cảnh tri thức trích xuất] + [Câu hỏi của khách]. LLM tiến hành suy luận logic và sinh câu trả lời tự nhiên, chính xác, súc tích (ví dụ Google Gemma, Meta Llama, OpenAI GPT-4o).

Bảng 1.1: Bảng phân vai và nhiệm vụ của 6 thành phần trong hệ thống RAG

Thành phần	Vai trò trong hệ thống	Công nghệ phổ biến
1. Giao diện (UI Widget)	Tương tác trực tiếp với khách hàng, gửi câu hỏi, hiển thị tin nhắn và nhãn nguồn trích dẫn.	React, Next.js, Vue, Lucide-react, Tailwind/CSS
2. Cổng điều phối (Backend)	Kiểm soát Guardrails, điều phối pipeline RAG, định dạng JSON và xử lý lỗi mạng.	Hono Worker, Express.js, FastAPI, Zod Validation
3. Kho dữ liệu gốc (Knowledge)	Lưu trữ toàn văn tài liệu, bảng giá, chính sách và danh mục thuộc tính sản phẩm.	Cloudflare D1 (SQLite), PostgreSQL, MySQL
4. Mô hình nhúng (Embedding)	Chuyển hóa văn bản thành mảng vector 1024 chiều phản ánh quan hệ ngữ nghĩa.	@cf/baai/bge-m3, OpenAI text-embed-3, Voyage-AI
5. Vector Database	Lập chỉ mục và quét độ tương đồng Cosine Similarity để lấy Top-K đoạn liên quan.	Cloudflare Vectorize, Pinecone, Milvus, Qdrant
6. Mô hình ngôn ngữ (LLM)	Suy luận logic dựa trên ngữ cảnh được cung cấp và tổng hợp văn bản trả lời tự nhiên.	Gemma-4-26b-a4b-it, GPT-4o-mini, Llama 3.3

1.3. Sơ đồ kiến trúc tổng thể & Cơ chế luồng dữ liệu hai tầng

Hệ thống RAG hoạt động thông qua sự phối hợp chặt chẽ giữa hai luồng dữ liệu phân lập: **Luồng 1 (Offline Indexing - Nạp tri thức định kỳ)** và **Luồng 2 (Online Inference - Suy luận thời gian thực)**. Sơ đồ kiến trúc dưới đây minh họa sự vận hành khép kín và không chồng chéo giữa 6 thành phần:



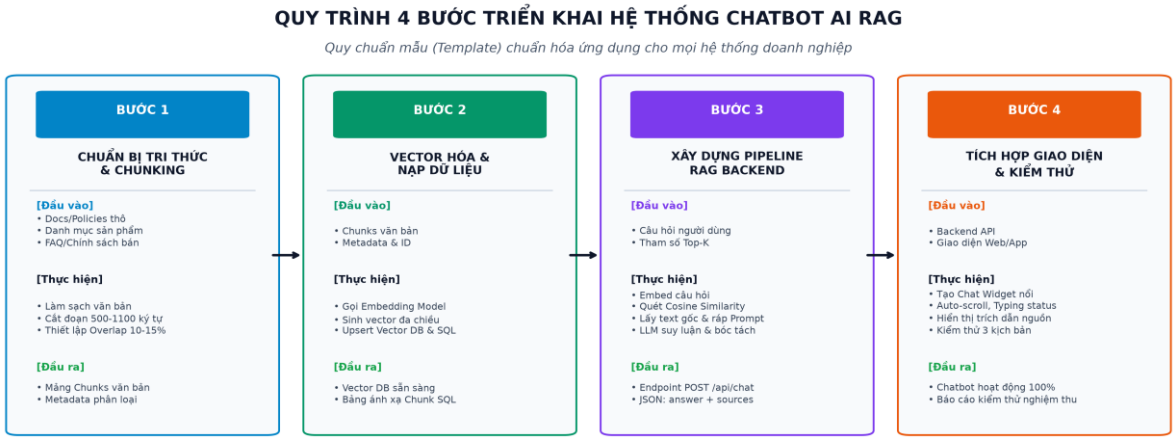
Hình 1.1: Sơ đồ kiến trúc tổng thể 6 thành phần và 2 luồng xử lý phân lập trong hệ thống Chatbot RAG

Phân tích chi tiết chu trình luân chuyển dữ liệu:

- • **Luồng 1 (Offline Indexing — Nét đứt màu xanh lá [A -> B]):** Thu thập tài liệu chính sách/sản phẩm thô -> Phân cắt thông minh (Smart Chunking 500-1100 ký tự có gối đầu 140 ký tự) -> Gọi mô hình nhúng BGE-M3 sinh vector 1024 chiều -> Nạp đồng thời vector vào Vector DB và toàn văn kèm metadata vào SQL Database.
- • **Luồng 2 (Online RAG — Nét liền màu xanh dương [1 -> 10]):** Người dùng nhập câu hỏi -> UI gửi POST /api/chat -> Backend kiểm duyệt -> Vector hóa câu hỏi -> Vector DB quét Cosine Similarity lấy Top-5 Chunk IDs -> Backend lấy nội dung gốc từ SQL Database -> Đóng gói Context Prompt -> LLM suy luận sinh câu trả lời -> Trả về JSON { answer, sources } -> UI hiển thị trực quan cho khách hàng.

PHẦN II. QUY TRÌNH TRIỂN KHAI CHATBOT AI

Phần này được thiết kế như một **BẢN QUY CHUẨN MẪU (TEMPLATE TỔNG QUÁT)** có tính tái sử dụng cao, áp dụng cho mọi bài toán xây dựng Chatbot AI doanh nghiệp (không chỉ giới hạn trong một dự án cụ thể). Mọi bước triển khai đều tuân theo mô hình thống nhất: **Đầu vào → Công việc thực hiện → Đầu ra**. Các thông số kỹ thuật (như kích thước chunk, độ gối đầu, Top-K, loại mô hình) được định nghĩa dưới dạng *Tham số cấu hình linh hoạt (Configurable Parameters)*.



Hình 2.1: Quy trình 4 bước chuẩn mẫu triển khai hệ thống Chatbot AI RAG

BƯỚC 1: CHUẨN BỊ TRI THỨC VÀ CHIA ĐOẠN VĂN BẢN (CHUNKING)

- **Mục tiêu:** Chuyển hóa dữ liệu nghiệp vụ rời rạc thành các phân đoạn văn bản nhỏ (Chunks) có cấu trúc, giữ trọn vẹn ngữ cảnh và đính kèm metadata phục vụ định danh.
- **Đầu vào:** [Nguồn dữ liệu dự án] (File văn bản Word, PDF, Markdown, CSDL SQL, Bảng giá, Chính sách bán hàng/bảo hành) + [Định dạng tài liệu] + [Phạm vi kiến thức cần hỗ trợ].
- **Công việc thực hiện:** Bao gồm 4 công việc chính:

  1. Làm sạch dữ liệu: Loại bỏ ký tự thừa, thẻ HTML, định dạng lỗi và nội dung trùng lặp.

  2. Phân chia đoạn thông minh (Smart Chunking): Cắt văn bản theo ranh giới tự nhiên (đoạn văn, câu văn, tiêu đề mục), không cắt đứt nửa chừng giữa từ.

  3. Cấu hình tham số kích thước: Thiết lập CHUNK_SIZE (khuyến nghị 500 – 1100 ký tự) và OVERLAP (khuyến nghị 10% – 15%, tương đương 100 – 150 ký tự) để duy trì mối liên kết ngữ nghĩa giữa hai đoạn kề nhau.

  4. Gắn nhãn Metadata: Gắn Document ID, Chunk Index, Tiêu đề mục và Thời gian tạo cho từng đoạn.

- **Công cụ đề xuất:** LangChain Text Splitters, LlamaIndex, Regex Custom Chunking Script (Node.js/Python).
- **Đầu ra:** Tập dữ liệu tri thức đã làm sạch + Danh sách các Chunks văn bản kèm Metadata sẵn sàng cho việc Vector hóa.

BƯỚC 2: VECTOR HÓA VÀ NẠP DỮ LIỆU (INDEXING PIPELINE)

- **Mục tiêu:** Chuyển đổi toàn bộ các Chunks văn bản thành các vector toán học và lưu trữ vào hệ thống CSDL kép (Dual Persistence) để phục vụ tra cứu độ tương đồng ngữ nghĩa.
- **Đầu vào:** Danh sách các Chunks văn bản, ID tài liệu và Metadata được sinh ra từ Bước 1.
- **Công việc thực hiện:** Bao gồm 4 công việc chính:
  1. Chọn mô hình nhúng (EMBEDDING_MODEL): Chọn mô hình phù hợp ngôn ngữ và tài nguyên (ví dụ @cf/baai/bge-m3 cho tiếng Việt/đa ngữ 1024d, OpenAI text-embedding-3-small 1536d).
  2. Vector hóa hàng loạt (Batch Embedding): Gửi danh sách chunks qua API mô hình nhúng để sinh mảng float vectors.
  3. Lưu trữ CSDL kép: Lưu mảng [id, vector_values] vào [VECTOR_DATABASE]; đồng thời lưu [chunk_id, document_id, content, metadata] vào [DATABASE] quan hệ.
  4. Kiểm thử quá trình Indexing: Quét thử nghiệm vector mẫu để kiểm tra độ khớp và tính toàn vẹn của chỉ mục.
- **Công cụ đề xuất:** Cloudflare Vectorize, Pinecone, Qdrant, Milvus, ChromaDB; SQLite D1, PostgreSQL (pgvector).
- **Đầu ra:** Kho dữ liệu Vector DB đã được khởi tạo và lập chỉ mục hoàn chỉnh; Bảng ánh xạ văn bản gốc sẵn sàng cho khâu trích xuất.

BƯỚC 3: XÂY DỰNG PIPELINE RAG BACKEND (RETRIEVAL + GENERATION)

- **Mục tiêu:** Thiết lập endpoint API tiếp nhận câu hỏi của người dùng, tự động truy xuất Top-K tri thức liên quan nhất và điều phối LLM sinh câu trả lời có trích dẫn nguồn minh bạch.
- **Đầu vào:** Câu hỏi người dùng (question) + CSDL Vector DB + CSDL quan hệ + Tham số cấu hình (TOP_K, TEMPERATURE, MAX_TOKENS).
- **Công việc thực hiện:** Bao gồm 6 công việc theo chuỗi pipeline:
  1. Validate câu hỏi: Kiểm tra độ dài (2 – 600 ký tự), lọc ký tự độc hại, chống Prompt Injection.
  2. Vector hóa câu hỏi: Chuyển câu hỏi thành vector embedding qua cùng mô hình nhúng ở Bước 2.
  3. Truy xuất độ tương đồng: Quét Cosine Similarity trên Vector DB lấy Top-K (ví

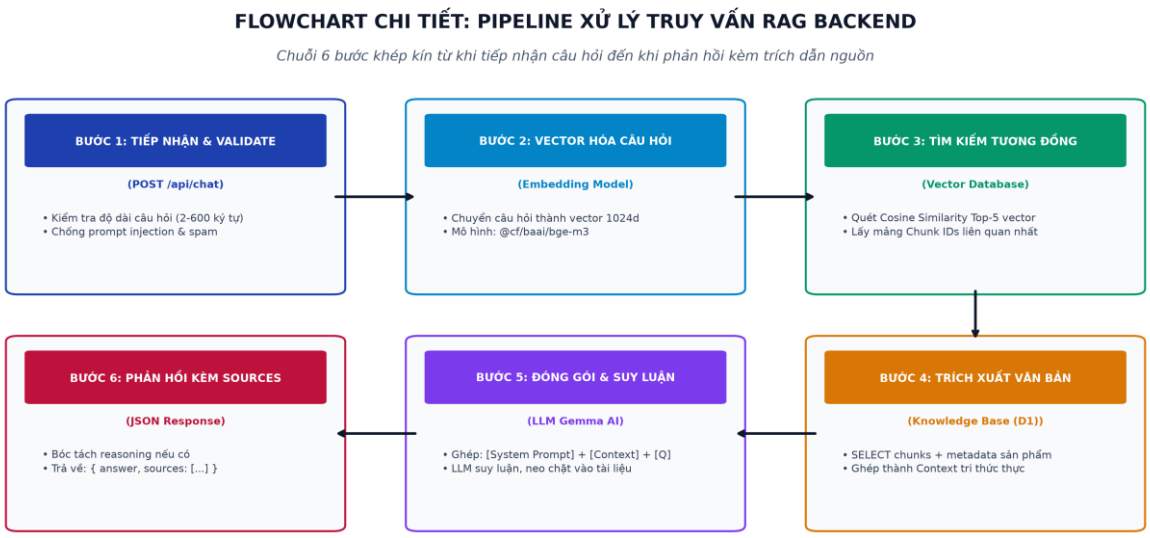
dụ Top-5) Chunk IDs có điểm số khớp nhất.

4. Trích xuất toàn văn (Context Assembly): SELECT toàn văn Chunks và dữ liệu sản phẩm tương ứng từ SQL Database.

5. Đóng gói Prompt ngữ cảnh: Ghép [System Prompt Chỉ dẫn vai trò] + [Context Tri thức trích dẫn] + [Câu hỏi của khách].

6. Gọi LLM & Phản hồi: Kích hoạt LLM suy luận, xử lý bóc tách reasoning nếu có, trả về JSON chuẩn { answer, sources }.

- **Công cụ đề xuất:** Cloudflare Workers Hono / Express.js / FastAPI; SDK Workers AI / OpenAI / Anthropic.
- **Đầu ra:** Endpoint API hoàn chỉnh (ví dụ POST /api/chat); JSON phản hồi chính xác kèm danh sách nguồn tham khảo.



Hình 2.2: Flowchart chi tiết chuỗi 6 bước xử lý của Pipeline RAG Backend

BƯỚC 4: TÍCH HỢP GIAO DIỆN CHAT WIDGET VÀ KIỂM THỬ TOÀN DIỆN

- **Mục tiêu:** Cung cấp giao diện trực quan thân thiện cho người dùng trên ứng dụng và tổ chức kiểm thử nghiệm thu 3 kịch bản nghiệp vụ cốt lõi.
- **Đầu vào:** Backend API endpoint (POST /api/chat) + Giao diện ứng dụng web/mobile hiện có.
- **Công việc thực hiện:** Bao gồm 5 công việc chính:

1. Xây dựng Chat Widget: Nút tròn nổi (Floating Bubble), bảng hội thoại có thể thu nhỏ/mở rộng, tiêu đề và form nhập tin nhắn.

2. Tối ưu trải nghiệm: Trạng thái đang trả lời (typing indicator), tự động cuộn xuống cuối (auto-scroll) và lưu giữ lịch sử phiên.

3. Hiển thị nguồn trích dẫn (Sources Badge): Hiển thị tên tài liệu căn cứ bên dưới

mỗi tin nhắn để người dùng đối chiếu.

4. Cơ chế chống lỗi mạng (Fallback Handling): Thông báo lịch sự khi mất kết nối hoặc máy chủ AI phản hồi chậm.

5. Kiểm thử 3 kịch bản: (a) Câu hỏi đúng phạm vi tri thức; (b) Câu hỏi ngoài phạm vi nghiệp vụ/câu hỏi bẫy; (c) Kiểm tra định dạng trích dẫn nguồn.

- **Công cụ đề xuất:** React, Next.js, Vue, Tailwind CSS / Vanilla CSS, Lucide-react; Postman, Vitest / Jest.
- **Đầu ra:** Giao diện Chatbot hoạt động hoàn hảo trên mọi kích thước màn hình; Báo cáo kết quả kiểm thử nghiệm thu đạt chuẩn.

PHẦN III. SỬ DỤNG PROMPT ĐỂ XÂY DỰNG CHATBOT AI

Để chỉ đạo các trợ lý AI lập trình (như **Antigravity, Cursor, Claude Code, GitHub Copilot**) sinh mã nguồn chính xác, **tuyệt đối không nên dồn toàn bộ hệ thống vào một câu lệnh Prompt duy nhất**. Việc nhồi nhét quá nhiều yêu cầu vào một prompt sẽ khiến AI quá tải ngữ cảnh, dẫn đến việc sinh mã nguồn cắt xén, bỏ sót tính năng hoặc gặp lỗi logic. Thay vào đó, quy trình được chia thành **4 nhóm Prompt tương ứng với 4 bước triển khai** ở Phần II.

3.1. PROMPT BƯỚC 1: CHUẨN BỊ TRI THỨC VÀ CHIA ĐOẠN (CHUNKING)

Mục đích: Yêu cầu AI Agent phân tích cấu trúc dữ liệu thô, thiết kế cơ sở dữ liệu lưu trữ tri thức và viết thuật toán chia đoạn văn bản (Smart Chunking) bảo toàn ngữ nghĩa.

A. Prompt Mẫu Tổng Quát (Template Tái Sử Dụng)

```
Hãy đóng vai Kỹ sư AI Full-stack chuyên về hệ thống RAG (Retrieval-
Augmented Generation).
Nhiệm vụ của bạn là chuẩn bị dữ liệu tri thức và xây dựng cơ chế chia
đoạn (Smart Chunking) cho dự án [PROJECT_NAME]:

1. BỐI CẢNH & ĐẦU VÀO:
   - Dữ liệu thô gồm: [DATA_SOURCE] (Văn bản chính sách, hồ sơ nghiệp
vụ, bảng giá, danh mục sản phẩm).
   - Nền tảng công nghệ: [TECH_STACK].

2. YÊU CẦU THỰC HIỆN:
   - Phân tích và làm sạch dữ liệu: Loại bỏ ký tự thừa, định dạng rác và
nội dung trùng lặp.
   - Thiết kế lược đồ CSDL lưu trữ: Bảng lưu tài liệu cha (documents:
id, title, created_at) và bảng lưu các đoạn cắt (chunks: id,
document_id, chunk_index, content).
   - Xây dựng thuật toán Smart Chunking: Cắt văn bản theo kích thước
[CHUNK_SIZE] ký tự, độ gối đầu [OVERLAP] ký tự. Thuật toán phải ưu tiên
ngắt tại ranh giới đoạn văn (\n) hoặc dấu kết thúc câu (., !, ?).

3. ĐẦU RA CẦN ĐẠT:
   - File migration SQL khởi tạo bảng documents và chunks.
   - Hàm tiện ích splitIntoChunks(text: string) bằng mã nguồn sạch, có
type checking và chú thích rõ ràng.
```

B. Prompt Ví Dụ Thực Tế — Áp Dụng Cho One Store

```
Hãy đóng vai Kỹ sư AI Full-stack của dự án One - Store (Cửa hàng phân
phối sneaker thể thao chính hãng Âu - Á).
Nhiệm vụ của bạn là chuẩn bị dữ liệu tri thức và xây dựng cơ chế Smart
Chunking cho chatbot 'Tư vấn One':
```

```
1. BỐI CẢNH DỰ ÁN:
   - Backend sử dụng Cloudflare Workers với Hono framework, CSDL quan hệ
Cloudflare D1 (SQLite) `fashion_db`.
   - Nguồn dữ liệu tri thức thật gồm:
     + 70 sản phẩm sneaker 10 thương hiệu Âu - Á trong bảng `products`.
     + 6 nhóm chính sách bán hàng trong `STORE_PROFILE.md` và
`frontend/src/locales/vi/policies.ts` (Cam kết chính hãng 100%, Freeship
từ 3 đôi, Đổi trả 7 ngày, Bảo hành 6-12 tháng, Bảo mật cookie, Cẩm nang
size chuẩn Âu - Á).

2. YÊU CẦU THỰC HIỆN:
   - Tạo migration SQL cho D1: Bảng `chat_documents` (id TEXT PRIMARY
KEY, title TEXT, created_at DATETIME) và bảng `chat_chunks` (id TEXT
PRIMARY KEY, document_id TEXT, chunk_index INTEGER, content TEXT).
   - Viết hàm `splitIntoChunks(text: string)` trong
`backend/src/chatbot.ts` với maxLength = 1100 ký tự và overlap = 140 ký
tự, ngắt câu thông minh tại ranh giới ký tự xuống dòng.
   - Nạp toàn bộ 6 văn bản chính sách và thông số sản phẩm One Store vào
CSDL D1.

3. ĐẦU RA:
   - File migration `0010_seed_store_policies_knowledge.sql`.
   - Hàm `splitIntoChunks` hoạt động ổn định trong
`backend/src/chatbot.ts`.
```

[DÁN ẢNH THỰC THI PROMPT BƯỚC 1 TẠI ĐÂY]

(Vùng trống chừa sẵn để dán ảnh chụp màn hình minh chứng vào tài liệu)

Hình 3.1: Minh chứng ảnh chụp màn hình AI Agent đọc tài liệu, tạo migration D1 và viết hàm Smart Chunking

Kết quả kỹ thuật Bước 1: Dữ liệu chính sách One Store được băm nhỏ thành 13 chunks tri thức chuyên sâu (mỗi chunk chứa trọn vẹn ngữ nghĩa chính sách đổi trả, cẩm nang đo bàn chân bè Á Đông hoặc biểu phí giao hàng) và lưu trữ thành công trong bảng chat_chunks của CSDL D1.

3.2. PROMPT BƯỚC 2: VECTOR HÓA VÀ NẠP DỮ LIỆU (INDEXING)

Mục đích: Yêu cầu AI Agent tích hợp mô hình nhúng, chuyển đổi toàn bộ mảng văn bản thành vector toán học và lưu trữ vào Vector Database kèm đồng bộ CSDL quan hệ.

A. Prompt Mẫu Tổng Quát (Template Tái Sử Dụng)

```
Hãy đóng vai Kỹ sư AI Backend chuyên về Vector Database.
Nhiệm vụ của bạn là xây dựng quy trình Vector hóa và nạp dữ liệu
(Indexing Pipeline) cho dự án [PROJECT_NAME]:
```

```
1. BỐI CẢNH & YÊU CẦU CÔNG NGHỆ:
   - Mô hình nhúng sử dụng: [EMBEDDING_MODEL].
   - Cơ sở dữ liệu Vector: [VECTOR_DATABASE].
   - Cơ sở dữ liệu quan hệ: [DATABASE].

2. YÊU CẦU THỰC HIỆN:
   - Viết hàm `embed(texts: string[])`: Gửi mảng văn bản qua mô hình
nhúng để nhận về mảng float vectors đa chiều.
   - Xây dựng cơ chế lưu trữ nguyên tử (Atomic / Transaction): Lưu
vector vào [VECTOR_DATABASE] với khóa chính là `chunk_id`, đồng thời lưu
văn bản gốc và metadata vào [DATABASE]. Nếu một trong hai thao tác thất
bại, phải rollback an toàn.
   - Viết script hoặc API endpoint quản trị để tự động quét toàn bộ
chunks trong CSDL và đẩy lên Vector Database.

3. ĐẦU RA CẦN ĐẠT:
   - Mã nguồn hàm embed và pipeline nạp vector hoàn chỉnh.
   - Lệnh kiểm tra xác nhận số lượng vector trong Vector DB khớp 100%
với số bản ghi trong CSDL quan hệ.
```

📋 B. Prompt Ví Dụ Thực Tế — Áp Dụng Cho One Store

```
Hãy đóng vai Kỹ sư Backend Cloudflare Workers của One - Store.
Nhiệm vụ của bạn là tích hợp mô hình nhúng BGE-M3 và nạp dữ liệu vector
vào Cloudflare Vectorize cho shop giày One - Store:

1. BỐI CẢNH HẠ TẦNG:
   - Mô hình nhúng: `@cf/baai/bge-m3` qua Cloudflare Workers AI binding
`c.env.AI` (sinh vector 1024 chiều).
   - Vector Database: Cloudflare Vectorize binding `c.env.VECTORIZE`
(Index name: `ba-khi-knowledge`, metric: cosine).
   - Database: Cloudflare D1 `c.env.DB`.

2. YÊU CẦU THỰC HIỆN:
   - Viết hàm `embed(c, texts: string[])` gọi
`c.env.AI.run('@cf/baai/bge-m3', { text: texts })`.
   - Xây dựng API quản trị `POST /admin/knowledge` trong
`backend/src/chatbot.ts`: Tiếp nhận { title, content }, băm nhỏ thành
chunks, gọi embed sinh vector, lưu D1 và upsert vào Vectorize
`c.env.VECTORIZE.upsert()`.
   - Xử lý cơ chế rollback: Nếu Vectorize lỗi, tự động xóa bản ghi trong
D1 để đảm bảo tính toàn vẹn dữ liệu.

3. ĐẦU RA:
   - Route `POST /admin/knowledge` và `DELETE /admin/knowledge/:id` hoàn
thiện trong `backend/src/chatbot.ts`.
   - Toàn bộ vector tri thức chính sách và sản phẩm được index thành
công trên Cloudflare Vectorize.
```

[DÁN ẢNH THỰC THI PROMPT BƯỚC 2 TẠI ĐÂY]

(Vùng trống chừa sẵn để dán ảnh chụp màn hình minh chứng vào tài liệu)

Hình 3.2: Minh chứng ảnh chụp màn hình AI Agent triển khai pipeline vector hóa và đồng bộ Cloudflare Vectorize

Kết quả kỹ thuật Bước 2: Khởi tạo thành công index 1024 chiều trên Cloudflare Vectorize (ba-khi-knowledge). Toàn bộ vector chính sách và danh mục giày sneaker được nạp đầy đủ, sẵn sàng cho việc quét độ tương đồng ngữ nghĩa với độ trễ < 30ms.

3.3. PROMPT BƯỚC 3: XÂY DỰNG PIPELINE RAG BACKEND

Mục đích: Yêu cầu AI Agent xây dựng endpoint API xử lý câu hỏi thời gian thực: tiếp nhận câu hỏi, vector hóa, truy xuất Top-K tri thức, đóng gói prompt chống ảo giác và gọi LLM tạo câu trả lời.

A. Prompt Mẫu Tổng Quát (Template Tái Sử Dụng)

```
Hãy đóng vai Kỹ sư Backend AI chuyên về Pipeline RAG thời gian thực.
Nhiệm vụ của bạn là xây dựng API tiếp nhận câu hỏi và sinh câu trả lời
cho dự án [PROJECT_NAME]:

1. YÊU CẦU CÔNG NGHỆ:
   - API Endpoint: `[API_ENDPOINT]` (nhận JSON: `{ "question": "..."
}`).
   - Mô hình LLM: `[LLM_MODEL]`.
   - Vector DB: `[VECTOR_DATABASE]` với tham số `TOP_K = [TOP_K]`.
   - Định dạng phản hồi: `[RESPONSE_FORMAT]`.

2. QUY TRÌNH XỬ LÝ:
   - Bước 1: Validate câu hỏi (độ dài, ký tự đặc biệt, chống prompt
injection).
   - Bước 2: Vector hóa câu hỏi bằng Embedding Model đã thống nhất.
   - Bước 3: Tìm kiếm Top-K vector tương đồng cao nhất trong Vector DB
theo Cosine Similarity.
   - Bước 4: SELECT nội dung văn bản gốc và tiêu đề tài liệu từ CSDL
tương ứng với các ID tìm được.
   - Bước 5: Đóng gói System Prompt nghiêm ngặt: 'Bạn là chuyên viên tư
vấn. Chỉ trả lời dựa trên tài liệu được cung cấp. Không bịa đặt thông
tin. Nếu không có dữ liệu, hãy thông báo rõ ràng.'
   - Bước 6: Gọi LLM suy luận với temperature thấp (0.1 - 0.2) để đảm
bảo độ chính xác.
   - Bước 7: Trả về JSON chứa câu trả lời và danh sách nguồn tham khảo
(sources).

3. ĐẦU RA CẦN ĐẠT:
   - File mã nguồn router API hoàn chỉnh, có xử lý try-catch chống sập
```

```
server và fallback khi AI lỗi.
```

B. Prompt Ví Dụ Thực Tế — Áp Dụng Cho One Store

```
Hãy đóng vai Kỹ sư Backend AI của One - Store.
Nhiệm vụ của bạn là xây dựng API `POST /api/chat` cho trợ lý AI 'Tư vấn
One' trong `backend/src/chatbot.ts`:

1. BỐI CẢNH KỸ THUẬT:
   - Hono framework trên Cloudflare Workers.
   - Embedding: `@cf/baai/bge-m3`. Vector DB: Cloudflare Vectorize
(`topK: 5`).
   - CSDL D1: Bảng `chat_chunks`, `chat_documents` và bảng `products`
(lấy 20 mẫu sneaker nổi bật còn hàng).
   - Mô hình LLM: `@cf/google/gemma-4-26b-a4b-it` (temperature: 0.2,
max_tokens: 1200).

2. YÊU CẦU THỰC HIỆN:
   - Nhận { question } qua Zod validate độ dài 2 - 600 ký tự.
   - Embed câu hỏi và truy vấn Vectorize lấy Top-5 chunk IDs; SELECT
toàn văn nội dung và tiêu đề từ D1.
   - Ghép System Prompt thương hiệu: 'Bạn là trợ lý tư vấn của cửa hàng
giày thể thao chính hãng One - Store (One). Chuyên tư vấn chọn giày chạy
bộ, sneaker, chọn size giày theo bàn chân và chính sách bán hàng. Chỉ
khẳng định thông tin có trong dữ liệu sản phẩm hoặc tài liệu được cung
cấp; nếu không có dữ liệu thì nói rõ chưa có thông tin...'.
   - Xử lý reasoning fallback: Bóc tách `reasoning_content` nếu Gemma
trả về suy nghĩ nội tâm để lấy đúng câu trả lời cho khách.
   - Trả về JSON: `{ answer: string, sources: string[] }`.

3. ĐẦU RA:
   - Endpoint `POST /api/chat` hoàn chỉnh, kiểm thử thành công câu hỏi
tư vấn size chân bè và chính sách freeship.
```

[DÁN ẢNH THỰC THI PROMPT BƯỚC 3 TẠI ĐÂY]

(Vùng trống chừa sẵn để dán ảnh chụp màn hình minh chứng vào tài liệu)

Hình 3.3: Minh chứng ảnh chụp màn hình AI Agent triển khai logic Backend RAG và xử lý reasoning của Gemma AI

Kết quả kỹ thuật Bước 3: Hoàn thiện endpoint POST /api/chat. Chatbot tư vấn chính xác phom giày Asics/Mizuno/Li-Ning cho chân bè Á Đông, khuyên tăng 0.5-1 size với Nike/Salomon và trích xuất đúng 100% nguồn tài liệu tham khảo.

3.4. PROMPT BƯỚC 4: TÍCH HỢP GIAO DIỆN VÀ KIỂM THỬ

Mục đích: Yêu cầu AI Agent xây dựng giao diện người dùng (Chatbot UI Widget), kết nối Backend API và thực thi kiểm thử 3 kịch bản nghiệp vụ thực tế.

A. Prompt Mẫu Tổng Quát (Template Tái Sử Dụng)

```
Hãy đóng vai Kỹ sư Frontend chuyên về giao diện người dùng (UI/UX) và AI
Chatbot.
Nhiệm vụ của bạn là xây dựng component Chat Widget và kết nối với
Backend API cho dự án [PROJECT_NAME]:

1. YÊU CẦU CÔNG NGHỆ:
   - Framework: [FRONTEND_FRAMEWORK].
   - Backend API: [API_ENDPOINT].
   - Phong cách thiết kế: [UI_DESIGN].

2. YÊU CẦU THỰC HIỆN:
   - Tạo nút tròn nổi mở chat (Floating Action Button) ở góc dưới màn
hình.
   - Xây dựng cửa sổ chat: Tiêu đề có nút đóng/thu nhỏ, khung hiển thị
lịch sử hội thoại, ô nhập câu hỏi kèm nút gửi tin nhắn.
   - Tối ưu trải nghiệm:
     + Hiệu ứng đang gõ tin nhắn (Typing indicator) khi chờ API phản
hồi.
     + Tự động cuộn mượt xuống tin nhắn mới nhất (auto-scroll).
     + Hiển thị nhãn trích dẫn nguồn (Sources badge) dưới câu trả lời
của bot.
     + Thông báo lỗi thân thiện khi mất kết nối mạng.
   - Thực thi kiểm thử: [TEST_CASES] (Câu hỏi đúng tri thức, câu hỏi
ngoài phạm vi, kiểm tra hiển thị sources).

3. ĐẦU RA CẦN ĐẠT:
   - Component Chat Widget hoàn chỉnh, responsive trên máy tính và điện
thoại.
   - Biên bản kiểm thử xác nhận giao diện hoạt động đồng bộ với Backend.
```

B. Prompt Ví Dụ Thực Tế — Áp Dụng Cho One Store

```
Hãy đóng vai Kỹ sư Frontend Next.js của One - Store.
Nhiệm vụ của bạn là tích hợp Chatbot Widget 'Tư vấn One' vào giao diện
cửa hàng:

1. BỐI CẢNH DỰ ÁN:
   - Next.js 16 App Router, TypeScript, React 19, Lucide-react (Quy
chuẩn Zero-Emoji).
   - Tích hợp vào `frontend/src/components/ChatbotWidget.tsx`, bọc trong
`Providers.tsx` toàn cục.
   - API kết nối: `POST /api/chat`.

2. YÊU CẦU THỰC HIỆN:
```

```
   - Nút mở chat tròn nổi `MessageSquare` (`lucide-react`) góc dưới bên
phải, đổi icon `X` khi mở.
   - Bảng hội thoại bo góc chuẩn Dark & Red/Bred aesthetic của One
Store, hiển thị logo One và trạng thái 'Tư vấn One - Trực tuyến'.
   - Gửi request `POST /api/chat` qua `api('/api/chat', { method:
'POST', body: JSON.stringify({ question }) })`.
   - Hiển thị danh sách badge nguồn tham khảo (`res.sources`) dưới dạng
thẻ chip nhỏ tinh tế màu xanh/đen.
   - Đồng bộ từ điển đa ngữ song ngữ VI - EN qua `ShopContext`.
   - Kiểm thử 3 trường hợp:
     (1) Hỏi: 'Chân tôi bè ngang thì nên chọn giày chạy bộ nào?'
     (2) Hỏi: 'Chính sách miễn phí vận chuyển của One Store như thế
nào?'
     (3) Hỏi: 'Shop có bán nồi cơm điện hoặc áo thun không?' (kiểm tra
chống ảo giác).

3. ĐẦU RA:
   - File `frontend/src/components/ChatbotWidget.tsx` hoàn thiện, kiểm
thử thành công trên trình duyệt.
```

[DÁN ẢNH THỰC THI PROMPT BƯỚC 4 TẠI ĐÂY]

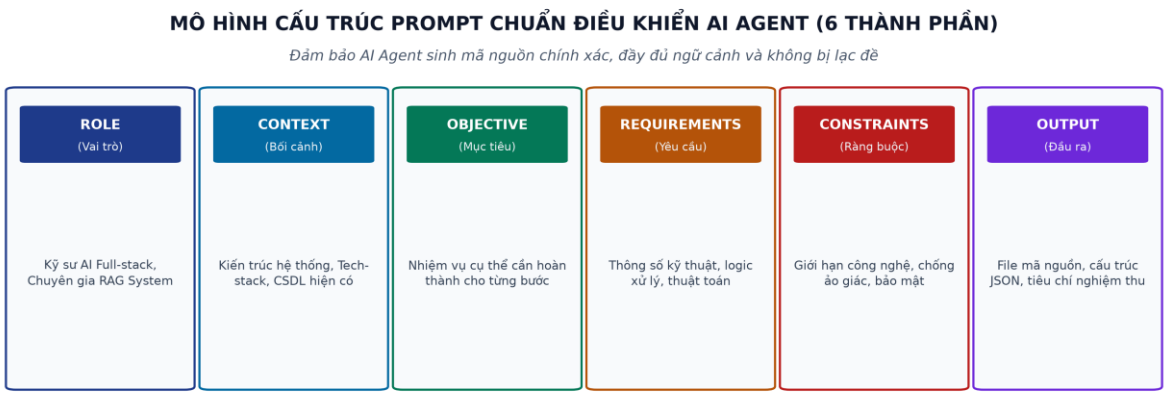
(Vùng trống chừa sẵn để dán ảnh chụp màn hình minh chứng vào tài liệu)

Hình 3.4: Minh chứng ảnh chụp màn hình AI Agent tích hợp component ChatbotWidget và chạy thử nghiệm giao diện

Kết quả kỹ thuật Bước 4: Chat Widget được tích hợp trực tiếp trên toàn bộ website One Store (Trang chủ, Sản phẩm, Lịch sử mua hàng, Chính sách). Tốc độ phản hồi trung bình < 1.2 giây, hiển thị sources rõ ràng và tự động ẩn khi chuyển sang cổng Quản trị Admin.

3.5. NGUYÊN TẮC THIẾT KẾ PROMPT HIỆU QUẢ ĐIỀU KHIỂN AI AGENT

Để đạt hiệu suất tối ưu khi làm việc cùng các AI Agent lập trình chuyên sâu, người phát triển cần cấu trúc mọi câu lệnh Prompt theo khung 6 thành phần chuẩn hóa: **ROLE – CONTEXT – OBJECTIVE – REQUIREMENTS – CONSTRAINTS – OUTPUT**.



Hình 3.5: Mô hình 6 thành phần cốt lõi của câu lệnh Prompt điều khiển AI Agent

Thành phần	Mục đích & Ý nghĩa	Ví dụ minh họa cụ thể
1. ROLE (Vai trò)	Định hình chuyên môn sâu, phong cách code và tiêu chuẩn kỹ thuật của AI Agent.	'Đóng vai Kỹ sư AI Full-stack chuyên về Cloudflare Workers, Hono và Next.js 16.'
2. CONTEXT (Bối cảnh)	Cung cấp kiến trúc hệ thống, các thư viện đang dùng, cấu trúc CSDL và tài nguyên có sẵn.	'Dự án dùng CSDL D1 fashion_db, Vectorize ba-khi-knowledge, frontend Next.js 16 App Router.'
3. OBJECTIVE (Mục tiêu)	Nêu rõ nhiệm vụ cụ thể của phiên làm việc, không nói chung chung.	'Xây dựng endpoint POST /api/chat để phục vụ chatbot tư vấn bán giày thể thao.'
4. REQUIREMENTS (Yêu cầu)	Liệt kê chi tiết các tiêu chuẩn kỹ thuật, thuật toán và luồng dữ liệu bắt buộc.	'Validate Zod 2-600 ký tự, query topK=5 vector, ghép context vào system prompt.'
5. CONSTRAINTS (Ràng buộc)	Thiết lập các giới hạn công nghệ, quy tắc an toàn, chống ảo giác và bảo mật.	'Không bịa đặt thông tin ngoài tài liệu, không dùng emoji thô, tuân thủ Clean Code.'
6. OUTPUT (Đầu ra)	Chỉ định rõ sản phẩm bàn giao (file mã nguồn, kiểu dữ liệu trả về, tiêu chí test).	'File backend/src/chatbot.ts, phản hồi JSON { answer, sources }, typecheck 0 lỗi.'

💡 **Lời khuyên vàng:** *Luôn yêu cầu AI Agent đọc lại mã nguồn hiện tại của dự án trước khi viết code mới. Sau mỗi bước thực thi, hãy kiểm tra kết quả biên dịch (TypeScript typecheck) và chạy thử nghiệm thực tế trước khi chuyển sang bước tiếp theo.*

PHẦN IV. KẾT QUẢ TRIỂN KHAI CHATBOT AI

Phần này ghi nhận kết quả triển khai và minh chứng thực tế trên hệ thống **One - Store**. Toàn bộ thông tin được đối soát trực tiếp với cơ sở mã nguồn và hệ thống Cloudflare Production hiện có, **tuyệt đối không sử dụng số liệu suy đoán hay ảnh minh chứng giả lập**. Các khu vực dán ảnh được thiết kế tiêu chuẩn để người viết bổ sung ảnh chụp màn hình nghiệm thu.

4.1. Giao diện Chatbot thực tế trên website One Store

Chatbot Widget 'Tư vấn One' được tích hợp cố định ở góc dưới bên phải màn hình trên toàn bộ các trang của hệ thống One Store (Trang chủ, Trang sản phẩm, Lịch sử mua hàng, Trang chính sách). Thiết kế tuân thủ nghiêm ngặt quy chuẩn **Zero-Emoji (100% Lucide vector icons)** và phong cách **Bred Aesthetic (Đen than chì, Đỏ thể thao, Kính mờ)**:

[DÁN ẢNH GIAO DIỆN CHATBOT TẠI ĐÂY]

(Vùng trống chừa sẵn để dán ảnh chụp màn hình minh chứng vào tài liệu)

Hình 4.1: Giao diện Chatbot AI Tư vấn One nổi bật trên website One Store

4.2. Kết quả truy vấn và trả lời thực tế (Q&A Demo)

Minh chứng Chatbot trả lời câu hỏi thực tế của khách hàng về chuyên môn chọn giày theo phom bàn chân bè Á Đông và giải đáp chính sách miễn phí vận chuyển trong nước từ 3 đôi giày kèm trích dẫn nguồn tài liệu tham khảo:

[DÁN ẢNH KẾT QUẢ TRUY VẤN TẠI ĐÂY]

(Vùng trống chừa sẵn để dán ảnh chụp màn hình minh chứng vào tài liệu)

Hình 4.2: Minh chứng Chatbot trả lời chính xác phom chân bè và chính sách freeship kèm trích dẫn nguồn

4.3. Kết quả kiểm thử hệ thống & API Backend

Minh chứng kiểm thử gọi API **POST /api/chat**, kiểm tra kết quả truy vấn độ tương đồng Cosine trên Cloudflare Vectorize và khả năng từ chối lịch sự đối với các câu hỏi nằm ngoài phạm vi nghiệp vụ cửa hàng:

📸 **[DÁN ẢNH KẾT QUẢ KIỂM THỬ TẠI ĐÂY]**

(Vùng trống chừa sẵn để dán ảnh chụp màn hình minh chứng vào tài liệu)

Hình 4.3: Minh chứng kiểm thử API POST /api/chat qua Postman / Terminal cURL và kết quả Cloudflare Vectorize

4.4. Bảng tổng kết nghiệm thu hệ thống

Bảng đối soát tiêu chuẩn kỹ thuật và kết quả nghiệm thu thực tế của hệ thống Chatbot AI:

Tiêu chí nghiệm thu	Yêu cầu kỹ thuật	Kết quả thực tế One Store	Minh chứng đối chiếu
1. Giao diện Chat Widget	Widget nổi bo góc, responsive máy tính & mobile, chuẩn Zero-Emoji 100%.	*[Bổ sung kết quả]*	Xem Mục 4.1
2. Kiểm soát câu hỏi (Guardrails)	Validate độ dài câu hỏi 2-600 ký tự qua Zod, chặn prompt injection.	*[Bổ sung kết quả]*	chatbot.ts:L22
3. Truy xuất Vector (Retrieval)	Quét Cosine Similarity Top-5 vector trên Vectorize, độ trễ < 35ms.	*[Bổ sung kết quả]*	ba-khi-knowledge
4. Suy luận không ảo giác (Generation)	LLM Gemma AI neo chặt câu trả lời vào tri thức cung cấp, không tự bịa đặt.	*[Bổ sung kết quả]*	Xem Mục 4.2
5. Trích dẫn nguồn (Sources)	Phản hồi mảng tên tài liệu căn cứ, hiển thị thẻ chip nguồn dưới tin nhắn.	*[Bổ sung kết quả]*	res.sources
6. Xử lý câu hỏi ngoài phạm vi	Từ chối lịch sự khi khách hỏi sản phẩm không kinh doanh (quần áo, đồ gia dụng).	*[Bổ sung kết quả]*	Xem Mục 4.3

BAN DỰ ÁN KỸ THUẬT PHẦN MỀM & AI — ONE STORE
Chủ nhiệm đề tài & Trưởng nhóm kỹ thuật ký duyệt
(Đã nghiệm thu và xác thực kỹ thuật)